CS 4510 Automata and Complexity 4/06/2025

Homework 9G: NP-Complete and Time Hierarchy

Devin Fromond *Due:4/13/2025*

You should submit a typeset or *neatly* written pdf on Gradescope. The grading TA should not have to struggle to read what you've written; if your handwriting is hard to decipher, you will be asked to typeset your future assignments. Four bonus points if you use LaTeX, and our template. You may collaborate with other students, but any written work should be your own. **Many of these problems have two sentence answers if you can apply the theorems proved in class**.

1. (20 points) Prove that the following problems are NP-complete.

   (a) (10 points) **Longest Path Problem.** Given an undirected graph $G = (V, E)$ and a positive integer $K \leq |V|$, does G contain a *simple path* (i.e., a path that visits no vertex more than once) with at least K edges?

   Answer: We first observe the Longest Path Problem is in NP since, given a path, we are able to verify the path is simple in poly-time.

   To show the Longest Path Problem is NP-Hard, we wish to reduce from the Hamiltonian Path problem. Given a Hamiltonian Path, we wish to set $K = |V| - 1$, where V are the vertices. We find a Hamiltonian Path is a simple path with $|V| - 1$ edges. Therefore, we are guaranteed an existence of a path which implies a corresponding answer to the Longest Path problem. Therefore, since the Hamiltonian Path problem is a known NP-Complete problem and we can perform this reduction in poly-time, we find the Longest Path Problem to be NP-Complete.

   (b) (10 points) **DOUBLE-SAT.**

   $$\text{DOUBLE-SAT} = \{\langle\varphi\rangle \mid \varphi \text{ is a Boolean formula with at least two satisfying assignments}\}.$$

   Answer: We observe the DOUBLE-SAT problem is in NP since we are able to verify that two distinct boolean assignments satisfy the formula in poly-time.

   To show the DOUBLE-SAT problem is NP-Hard, we wish to reduce from SAT. Given the boolean formula φ, we wish to construct a similar formula φ' by adding a variable z as a *dummy variable* or as a trivial clause. In other words, we wish to let $\varphi' = \varphi \wedge (z \vee \neg z)$. We find that if φ is satisfied, then φ' is satisfied. Additionally, for every satisfying assignment, there exists two assignments (z and $\neg z$) which ensures φ' has at least two satisfying assignments. It is trivial that the opposite is true as well. Therefore, we observe deciding whether φ' has two satisfying assignments decides the SAT problem, which we know to be NP-Complete. Therefore, we find this establishes DOUBLE-SAT as NP-Hard, but since DOUBLE-SAT is also in NP, we find DOUBLE-SAT is NP-Complete.

2. (10 points) **Time Hierarchy Theorem.**

   Let TIME($f(n)$) denote the class of languages decidable by a deterministic Turing machine in $O(f(n))$ time.

   Choose the correct relationship between the following two time complexity classes and prove your answer using asymptotic analysis and the Time Hierarchy Theorem:

   (a) $\text{TIME}(2^{n^2}) = \text{TIME}(n^4 \cdot 2^n)$
   (b) $\text{TIME}(2^{n^2}) \subset \text{TIME}(n^4 \cdot 2^n)$
   (c) $\text{TIME}(n^4 \cdot 2^n) \subset \text{TIME}(2^{n^2})$

   Answer: The correct relationship is $TIME(n^4 2^n)$ is strictly contained in $TIME(2^{n^2})$. We are able to prove this correctness by taking log on both sides yielding n^2 verses $4log_2 n + n$. We trivially find n^2 grows faster than $4log_2 n + n$. Therefore, we find any language decidable in time $n^4 2n$ can be decided in time 2^{n^2}, but not the reverse.

   When applying the Time Hierarchy Thm., we find that $f(n) = n^4 2^n$ and $g(n) = 2^{n^2}$. We find the condition of $f(n) = o(\frac{g(n)}{log(g(n))})$ holds since $n^4 2^n$ is $o(\frac{2^{n^2}}{n^2})$. Therefore, we similarly find there are languages decidable in 2^{n^2} time that cannotbe decided in $n^4 2^n$ which establishes $\text{TIME}(n^4 \cdot 2^n) \subset \text{TIME}(2^{n^2})$.

3. (10 points) **Savitch's Theorem.**

   Savitch's Theorem states that for any space-constructable function $f(n) \geq \log n$,

   $$\text{NSPACE}(f(n)) \subseteq \text{DSPACE}(f(n)^2).$$

   (a) (5 points) Use Savitch's Theorem to show that $\text{NL} \subseteq \text{DSPACE}((\log n)^2)$.
   (b) (5 points) Does Savitch's Theorem imply that L = NL? Briefly explain why or why not.

   Answer: We begin by letting $f(n) = logn$, we observe $NL = NSPACE(logn) \subseteq DSPACE((logn)^2)$. Consequently, we find every language that can be decided in non-deterministic log-space can be decided using deterministic space proportional to $(logn)^2$.

   One should note this does not imply $L = NL$ since the deterministic simulation requires more space than the $O(logn)$ bound which defines L. Therefore, we while we can simulate non-deterministic log-pace with slightly larger deterministic space, the gap between $O(logn)$ and $O((logn)^2)$ is large enough to not establish equality.